Session 22 (AIML) - Optional Task

Name**: Vaishnavi Praveen Rathi**
College**: Zeal Polytechnic, Narhe, Pune**
Branch**: Artificial Intelligence and Machine Learning (AIML)**
Domain**: AIML (Artificial Intelligence and Machine Learning)**

Github Main Repo Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments

Github File Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Session22/Session22%20OptionalTask/Assignment22%20OptionalTask.ipynb

Dataset Used: – Kaggle - https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud?

GitHub Repository: https://github.com/vaishnavi-rathiii/credit-fraud-Streamlit

Deployed Streamlit App: https://credit-fraud-app-7xoz6ujzdsxuydxsapdaps.streamlit.app/

Program Output :

1. Data Loading & Preprocessing

[1]:
```
'''
Data Loading & Preprocessing

Load the Credit Card Fraud Detection dataset.
Perform basic preprocessing and separate features and target.
'''

import pandas as pd

# Load the dataset
df = pd.read_csv("creditcard.csv")

# Display first 5 rows
print("First 5 Rows of Dataset:")
print(df.head())

# Display dataset shape
print("\nDataset Shape:", df.shape)

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Check duplicate rows
print("\nDuplicate Rows:", df.duplicated().sum())

# Remove duplicate rows
df = df.drop_duplicates()

print("\nDataset Shape After Removing Duplicates:", df.shape)

# Separate Features and Target
X = df.drop("Class", axis=1)
y = df["Class"]

# Display shapes
print("\nShape of X:", X.shape)
print("Shape of y:", y.shape)

# Display statistical summary
print("\nStatistical Summary:")
print(df.describe())
```

```
First 5 Rows of Dataset:
   Time        V1        V2        V3        V4        V5        V6        V7  \
0   0.0 -1.359807 -0.072781  2.536347  1.378155 -0.338321  0.462388  0.239599
1   0.0  1.191857  0.266151  0.166480  0.448154  0.060018 -0.082361 -0.078803
2   1.0 -1.358354 -1.340163  1.773209  0.379780 -0.503198  1.800499  0.791461
3   1.0 -0.966272 -0.185226  1.792993 -0.863291 -0.010309  1.247203  0.237609
4   2.0 -1.158233  0.877737  1.548718  0.403034 -0.407193  0.095921  0.592941

         V8        V9  ...       V21       V22       V23       V24       V25  \
0  0.098698  0.363787  ... -0.018307  0.277838 -0.110474  0.066928  0.128539
1  0.085102 -0.255425  ... -0.225775 -0.638672  0.101288 -0.339846  0.167170
2  0.247676 -1.514654  ...  0.247998  0.771679  0.909412 -0.689281 -0.327642
3  0.377436 -1.387024  ... -0.108300  0.005274 -0.190321 -1.175575  0.647376
4 -0.270533  0.817739  ... -0.009431  0.798278 -0.137458  0.141267 -0.206010

        V26       V27       V28  Amount  Class
0 -0.189115  0.133558 -0.021053  149.62      0
1  0.125895 -0.008983  0.014724    2.69      0
2 -0.139097 -0.055353 -0.059752  378.66      0
3 -0.221929  0.062723  0.061458  123.50      0
4  0.502292  0.219422  0.215153   69.99      0

[5 rows x 31 columns]

Dataset Shape: (284807, 31)
```

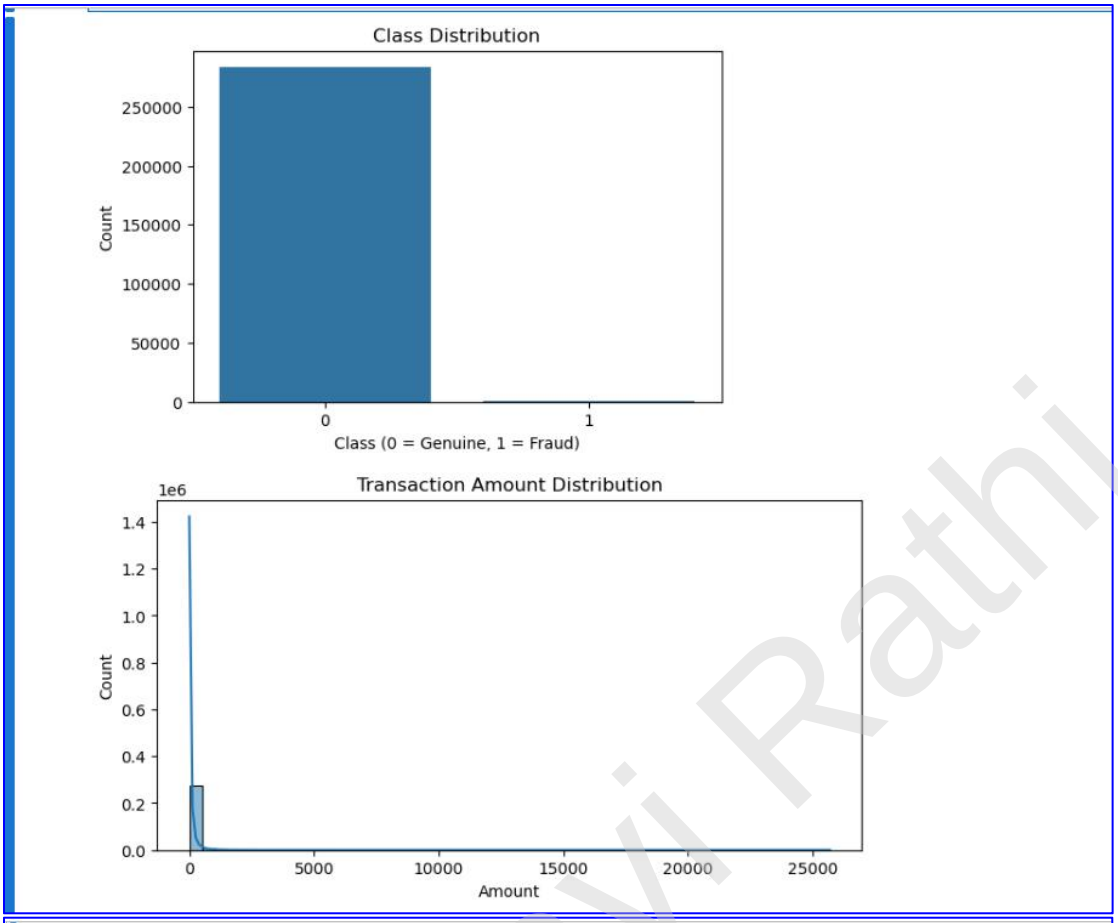
```
Missing Values:
Time      0
V1        0
V2        0
V3        0
V4        0
V5        0
V6        0
V7        0
V8        0
V9        0
V10       0
V11       0
V12       0
V13       0
V14       0
V15       0
V16       0
V17       0
V18       0
V19       0
V20       0
V21       0
V22       0
V23       0
V24       0
V25       0
V26       0
V27       0
V28       0
Amount    0
Class     0
```

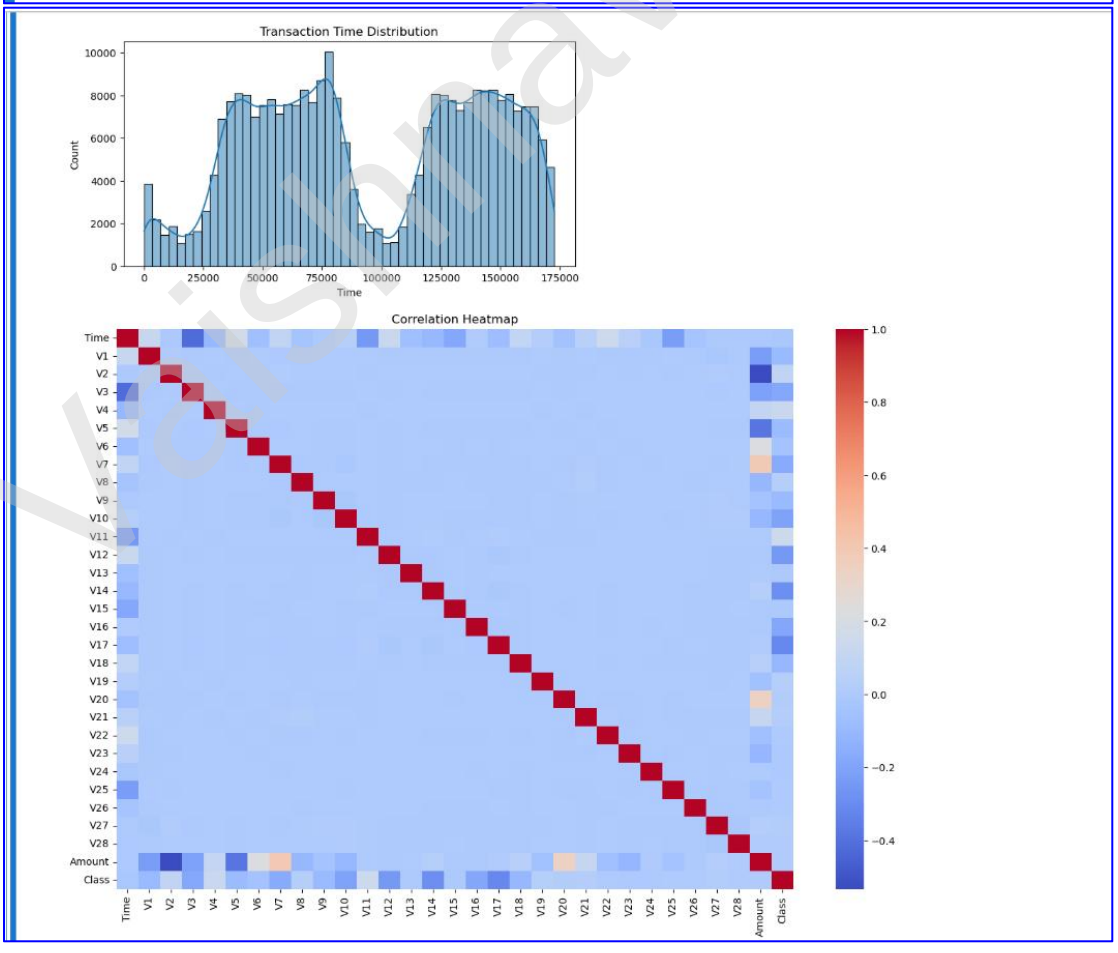
```
dtype: int64

Duplicate Rows: 1081

Dataset Shape After Removing Duplicates: (283726, 31)

Shape of X: (283726, 30)
Shape of y: (283726,)

Statistical Summary:
                Time             V1             V2             V3  \
count  283726.000000  283726.000000  283726.000000  283726.000000
mean    94811.077600       0.005917      -0.004135       0.001613
std     47481.047891       1.948026       1.646703       1.508682
min         0.000000     -56.407510     -72.715728     -48.325589
25%     54204.750000      -0.915951      -0.600321      -0.889682
50%     84692.500000       0.020384       0.063949       0.179963
75%    139298.000000       1.316068       0.800283       1.026960
max    172792.000000       2.454930      22.057729       9.382558

                 V4             V5             V6             V7  \
count  283726.000000  283726.000000  283726.000000  283726.000000
mean       -0.002966       0.001828      -0.001139       0.001801
std         1.414184       1.377008       1.331931       1.227664
min        -5.683171    -113.743307     -26.160506     -43.557242
25%        -0.850134      -0.689830      -0.769031      -0.552509
50%        -0.022248      -0.053468      -0.275168       0.040859
75%         0.739647       0.612218       0.396792       0.570474
max        16.875344      34.801666      73.301626     120.589494

                 V8             V9  ...            V21            V22  \
count  283726.000000  283726.000000  ...  283726.000000  283726.000000
mean       -0.000854      -0.001596  ...      -0.000371      -0.000015
std         1.179054       1.095492  ...       0.723909       0.724550
min       -73.216718     -13.434066  ...     -34.830382     -10.933144
25%        -0.208828      -0.644221  ...      -0.228305      -0.542700
50%         0.021898      -0.052596  ...      -0.029441       0.006675
75%         0.325704       0.595977  ...       0.186194       0.528245
max        20.007208      15.594995  ...      27.202839      10.503090

                V23            V24            V25            V26  \
count  283726.000000  283726.000000  283726.000000  283726.000000
mean        0.000198       0.000214      -0.000232       0.000149
std         0.623702       0.605627       0.521220       0.482053
min       -44.807735      -2.836627     -10.295397      -2.604551
25%        -0.161703      -0.354453      -0.317485      -0.326763
50%        -0.011159       0.041016       0.016278      -0.052172
75%         0.147748       0.439738       0.350667       0.240261
max        22.528412       4.584549       7.519589       3.517346

                V27            V28         Amount          Class
count  283726.000000  283726.000000  283726.000000  283726.000000
mean        0.001763       0.000547      88.472687       0.001667
std         0.395744       0.328027     250.399437       0.040796
min       -22.565679     -15.430084       0.000000       0.000000
25%        -0.070641      -0.052818       5.600000       0.000000
50%         0.001479       0.011288      22.000000       0.000000
75%         0.091208       0.078276      77.510000       0.000000
max        31.612198      33.847808   25691.160000       1.000000

[8 rows x 31 columns]
```

2. Exploratory Data Analysis (EDA)

[2]:
```
'''
Exploratory Data Analysis (EDA)
'''

import matplotlib.pyplot as plt
import seaborn as sns

# Class Distribution
plt.figure(figsize=(6,4))
sns.countplot(x="Class", data=df)
plt.title("Class Distribution")
plt.xlabel("Class (0 = Genuine, 1 = Fraud)")
plt.ylabel("Count")
plt.show()

# Transaction Amount Distribution
plt.figure(figsize=(8,4))
sns.histplot(df["Amount"], bins=50, kde=True)
plt.title("Transaction Amount Distribution")
plt.xlabel("Amount")
plt.show()

# Transaction Time Distribution
plt.figure(figsize=(8,4))
sns.histplot(df["Time"], bins=50, kde=True)
plt.title("Transaction Time Distribution")
plt.xlabel("Time")
plt.show()
```

```
# Correlation Heatmap
plt.figure(figsize=(15,10))
sns.heatmap(df.corr(), cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```





3. Feature Scaling

[3]:
```
'''
Feature Scaling
'''

from sklearn.preprocessing import StandardScaler

# Create StandardScaler object
scaler = StandardScaler()

# Scale only Time and Amount columns
X[["Time", "Amount"]] = scaler.fit_transform(X[["Time", "Amount"]])

print("Feature Scaling Completed Successfully.")
```

```
Feature Scaling Completed Successfully.
```

4. Train-Test Split

[4]:
```
'''
Train-Test Split
'''

from sklearn.model_selection import train_test_split

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

# Display shapes
print("X_train Shape:", X_train.shape)
print("X_test Shape :", X_test.shape)
print("y_train Shape:", y_train.shape)
print("y_test Shape :", y_test.shape)
```

```
X_train Shape: (226980, 30)
X_test Shape : (56746, 30)
y_train Shape: (226980,)
y_test Shape : (56746,)
```

5. Build and Train Logistic Regression Model

[7]:
```
'''
Build and Train Logistic Regression Model
'''

from sklearn.linear_model import LogisticRegression

# Create Logistic Regression model
model = LogisticRegression(max_iter=1000, random_state=42)

# Train the model
model.fit(X_train, y_train)

print("Logistic Regression model trained successfully.")
```

```
Logistic Regression model trained successfully.
```

6. Make Predictions on Test Data

[8]:
```
'''
Make Predictions on Test Data
'''

# Predict on test data
y_pred = model.predict(X_test)

# Display first 10 actual and predicted values
comparison = pd.DataFrame({
    "Actual": y_test.values[:10],
    "Predicted": y_pred[:10]
})

print(comparison)
```

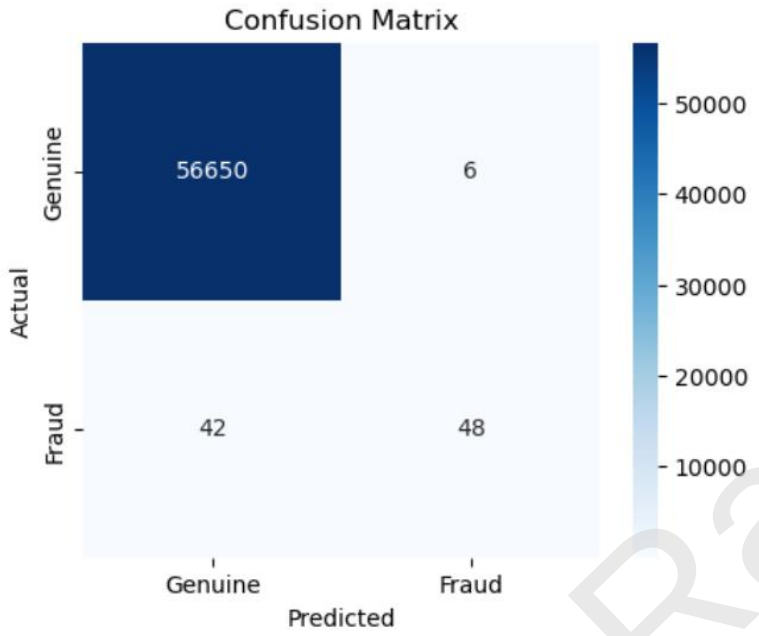
```
   Actual  Predicted
0       0          0
1       0          0
2       0          0
3       0          0
4       0          0
5       0          0
6       0          0
7       0          0
8       0          0
9       0          0
```

7. Confusion Matrix

[9]:
```
'''
Confusion Matrix
'''

from sklearn.metrics import confusion_matrix

# Generate confusion matrix
cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(cm)

# Display heatmap
plt.figure(figsize=(5,4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Genuine", "Fraud"],
    yticklabels=["Genuine", "Fraud"]
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

```
# Display TN, FP, FN, TP
TN, FP, FN, TP = cm.ravel()

print("True Negatives (TN):", TN)
print("False Positives (FP):", FP)
print("False Negatives (FN):", FN)
print("True Positives (TP):", TP)
```

```
Confusion Matrix:
[[56650     6]
 [   42    48]]
```




```
True Negatives (TN): 56650
False Positives (FP): 6
False Negatives (FN): 42
True Positives (TP): 48
```

8. Model Evaluation

[10]:
```
'''
Model Evaluation
'''

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

# Calculate evaluation metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1 Score : {f1:.4f}")

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

```
Accuracy : 0.9992
Precision: 0.8889
Recall   : 0.5333
F1 Score : 0.6667

Classification Report:

              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56656
           1       0.89      0.53      0.67        90

    accuracy                           1.00     56746
   macro avg       0.94      0.77      0.83     56746
weighted avg       1.00      1.00      1.00     56746
```

9. Save Model and Preprocessing Objects

[11]:
```
'''
Save Model and Preprocessing Objects
'''

import joblib

# Save model
joblib.dump(model, "fraud_model.pkl")

# Save scaler
joblib.dump(scaler, "scaler.pkl")

# Save feature columns
joblib.dump(X.columns.tolist(), "columns.pkl")

print("Model and preprocessing objects saved successfully.")
```

```
Model and preprocessing objects saved successfully.
```

10. Deployment